Homestay Booking & Reviews

Java Programming Project — Source Code

Project overview

A Java console application for managing homestay listings, hosts, guests, bookings, payments, cancellations, availability, and reviews. Student name, register number, and page-number labels have been omitted from this document.

How to compile

```
javac -d out src/model/*.java src/payment/*.java src/service/*.java src/app/*.java
java -cp out app.Main
```

Java source files

src/model/Person.java

```
package model;

public abstract class Person {
    private int id;
    private String name;
    private String phone;

    public Person(int id, String name, String phone) {
        this.id = id;
        this.name = name;
        this.phone = phone;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getPhone() { return phone; }
}
```

src/model/Host.java

```
package model;

public class Host extends Person {
    public Host(int id, String name, String phone) {
        super(id, name, phone);
    }

    @Override
    public String toString() {
        return "Host ID: " + getId() + ", Name: " + getName()
                + ", Phone: " + getPhone();
    }
}
```

src/model/Guest.java

```
package model;

public class Guest extends Person {
    public Guest(int id, String name, String phone) {
        super(id, name, phone);
    }

    @Override
    public String toString() {
        return "Guest ID: " + getId() + ", Name: " + getName()
                + ", Phone: " + getPhone();
    }
}
```

src/model/Listing.java

```
package model;

public class Listing {
    private int id;
    private String name;
    private String location;
    private double price;
    private Host host;

    public Listing(int id, String name, String location, double price, Host host) {
        this.id = id;
        this.name = name;
        this.location = location;
        this.price = price;
        this.host = host;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getLocation() { return location; }
    public double getPrice() { return price; }
    public Host getHost() { return host; }

    @Override
    public String toString() {
        return "Listing ID: " + id + ", Name: " + name
                + ", Location: " + location
                + ", Price/Night: Rs." + price
                + ", Host: " + host.getName();
    }
}
```

src/model/Booking.java

```
package model;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

public class Booking {
    private int id;
    private Listing listing;
    private Guest guest;
    private LocalDate checkIn;
    private LocalDate checkOut;
    private String status;
    private double amount;

    public Booking(int id, Listing listing, Guest guest,
                   LocalDate checkIn, LocalDate checkOut) {
        this.id = id;
        this.listing = listing;
        this.guest = guest;
        this.checkIn = checkIn;
        this.checkOut = checkOut;
        this.amount = ChronoUnit.DAYS.between(checkIn, checkOut) * listing.getPrice();
        this.status = "PENDING_PAYMENT";
    }

    public int getId() { return id; }
    public Listing getListing() { return listing; }
    public Guest getGuest() { return guest; }
    public LocalDate getCheckIn() { return checkIn; }
    public LocalDate getCheckOut() { return checkOut; }
    public String getStatus() { return status; }
    public double getAmount() { return amount; }
    public void setStatus(String status) { this.status = status; }

    public boolean overlaps(LocalDate start, LocalDate end) {
        return checkIn.isBefore(end) && start.isBefore(checkOut);
    }

    @Override
    public String toString() {
        return "Booking ID: " + id
                + ", Listing: " + listing.getName()
                + ", Host: " + listing.getHost().getName()
                + ", Guest: " + guest.getName()
                + ", Dates: " + checkIn + " to " + checkOut
                + ", Amount: Rs." + amount
                + ", Status: " + status;
    }
}
```

src/model/Review.java

```
package model;

public class Review {
    private Booking booking;
    private int rating;
    private String comment;

    public Review(Booking booking, int rating, String comment) {
        this.booking = booking;
        this.rating = rating;
        this.comment = comment;
    }

    public Booking getBooking() { return booking; }
    public int getRating() { return rating; }

    @Override
    public String toString() {
        return "Listing: " + booking.getListing().getName()
                + ", Guest: " + booking.getGuest().getName()
                + ", Rating: " + rating + "/5"
                + ", Review: " + comment;
    }
}
```

src/payment/Payment.java

```
package payment;

public abstract class Payment {
    private int bookingId;
    private double amount;

    public Payment(int bookingId, double amount) {
        this.bookingId = bookingId;
        this.amount = amount;
    }

    public int getBookingId() { return bookingId; }
    public double getAmount() { return amount; }
    public abstract String getMethod();

    @Override
    public String toString() {
        return "Booking ID: " + bookingId + ", Amount: Rs."
                + amount + ", Method: " + getMethod();
    }
}
```

src/payment/CashPayment.java

```
package payment;

public class CashPayment extends Payment {
    public CashPayment(int bookingId, double amount) {
        super(bookingId, amount);
    }

    @Override
    public String getMethod() {
        return "Cash";
    }
}
```

src/payment/UpiPayment.java

```
package payment;

public class UpiPayment extends Payment {
    public UpiPayment(int bookingId, double amount) {
        super(bookingId, amount);
    }

    @Override
    public String getMethod() {
        return "UPI";
    }
}
```

src/service/Cancellation.java

```
package service;

public class Cancellation {
    private int bookingId;
    private double fee;
    private String reason;

    public Cancellation(int bookingId, double fee, String reason) {
        this.bookingId = bookingId;
        this.fee = fee;
        this.reason = reason;
    }

    @Override
    public String toString() {
        return "Booking ID: " + bookingId
                + ", Fee: Rs." + fee
                + ", Reason: " + reason;
    }
}
```

src/service/HomestayService.java

```
package service;

import model.*;
import payment.*;
import java.util.ArrayList;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

public class HomestayService {
    private ArrayList<Host> hosts = new ArrayList<>();
    private ArrayList<Guest> guests = new ArrayList<>();
    private ArrayList<Listing> listings = new ArrayList<>();
    private ArrayList<Booking> bookings = new ArrayList<>();
    private ArrayList<Payment> payments = new ArrayList<>();
    private ArrayList<Cancellation> cancellations = new ArrayList<>();
    private ArrayList<Review> reviews = new ArrayList<>();

    private int hostId = 1, guestId = 1;
    private int listingId = 1, bookingId = 1;

    public void addHost(String name, String phone) {
        hosts.add(new Host(hostId++, name, phone));
        System.out.println("Host added successfully.");
    }

    public void addGuest(String name, String phone) {
        guests.add(new Guest(guestId++, name, phone));
        System.out.println("Guest added successfully.");
    }

    public void addListing(String name, String location, double price, int hId) {
        Host host = null;
        for (Host h : hosts) {
            if (h.getId() == hId) host = h;
        }
        if (host == null || price <= 0) {
            System.out.println("Invalid host ID or price.");
            return;
        }
        listings.add(new Listing(listingId++, name, location, price, host));
        System.out.println("Listing added successfully.");
    }

    private Listing findListing(int id) {
        for (Listing l : listings) if (l.getId() == id) return l;
        return null;
    }

    private Guest findGuest(int id) {
        for (Guest g : guests) if (g.getId() == id) return g;
        return null;
    }

    private Booking findBooking(int id) {
        for (Booking b : bookings) if (b.getId() == id) return b;
        return null;
    }

    public boolean isAvailable(int id, LocalDate in, LocalDate out) {
        if (findListing(id) == null || in == null || out == null
                || !in.isBefore(out) || in.isBefore(LocalDate.now())) return false;

        for (Booking b : bookings) {
            if (b.getListing().getId() == id
                    && (b.getStatus().equals("PENDING_PAYMENT")
                    || b.getStatus().equals("CONFIRMED"))
                    && b.overlaps(in, out)) return false;
        }
        return true;
    }

    public void createBooking(int lId, int gId, LocalDate in, LocalDate out) {
        Listing listing = findListing(lId);
        Guest guest = findGuest(gId);
        if (listing == null || guest == null || !isAvailable(lId, in, out)) {
            System.out.println("Invalid details or unavailable dates.");
            return;
        }
        Booking b = new Booking(bookingId++, listing, guest, in, out);
```

```
        bookings.add(b);
        System.out.println("Booking created: " + b);
        System.out.println("Record payment to confirm the booking.");
    }

    public void recordPayment(int id, int method) {
        Booking b = findBooking(id);
        if (b == null || !b.getStatus().equals("PENDING_PAYMENT")) {
            System.out.println("No pending booking found.");
            return;
        }

        Payment p;
        if (method == 1) p = new CashPayment(id, b.getAmount());
        else if (method == 2) p = new UpiPayment(id, b.getAmount());
        else {
            System.out.println("Invalid payment method.");
            return;
        }

        payments.add(p);
        b.setStatus("CONFIRMED");
        System.out.println("Payment successful: " + p);
        System.out.println("Booking confirmed: " + b);
    }

    public void cancelBooking(int id, String reason) {
        Booking b = findBooking(id);
        if (b == null || !(b.getStatus().equals("CONFIRMED")
                || b.getStatus().equals("PENDING_PAYMENT"))) {
            System.out.println("Booking cannot be cancelled.");
            return;
        }

        long days = ChronoUnit.DAYS.between(LocalDate.now(), b.getCheckIn());
        double rate = days >= 7 ? 0.10 : days >= 2 ? 0.30 : 0.80;
        double fee = b.getStatus().equals("CONFIRMED") ? b.getAmount() * rate : 0;
        double refund = b.getStatus().equals("CONFIRMED") ? b.getAmount() - fee : 0;
        b.setStatus("CANCELLED");
        cancellations.add(new Cancellation(id, fee, reason));
        System.out.println("Booking cancelled. Total: Rs." + b.getAmount()
                + ", Cancellation fee: Rs." + fee + ", Refund: Rs." + refund);
    }

    public void addReview(int id, int rating, String comment) {
        Booking b = findBooking(id);
        if (b == null || !b.getStatus().equals("CONFIRMED")
                || LocalDate.now().isBefore(b.getCheckOut())) {
            System.out.println("Review allowed only after completed stays.");
            return;
        }
        if (rating < 1 || rating > 5) {
            System.out.println("Rating must be between 1 and 5.");
            return;
        }
        for (Review r : reviews) {
            if (r.getBooking() == b) {
                System.out.println("Review already submitted.");
                return;
            }
        }
        b.setStatus("COMPLETED");
        reviews.add(new Review(b, rating, comment));
        System.out.println("Review added successfully.");
    }

    public void displayAvailability(LocalDate in, LocalDate out) {
        if (in == null || out == null || !in.isBefore(out)) {
            System.out.println("Invalid dates.");
            return;
        }
        for (Listing l : listings) {
            System.out.println(l + " | "
                    + (isAvailable(l.getId(), in, out) ? "AVAILABLE" : "UNAVAILABLE"));
        }
    }

    public void displayRecords() {
        System.out.println("\n--- HOSTS ---");
        for (Host h : hosts) System.out.println(h);
        System.out.println("\n--- GUESTS ---");
        for (Guest g : guests) System.out.println(g);
```

```
        System.out.println("\n--- LISTINGS ---");
        for (Listing l : listings) System.out.println(l);
        System.out.println("\n--- BOOKINGS ---");
        for (Booking b : bookings) System.out.println(b);
        System.out.println("\n--- PAYMENTS ---");
        for (Payment p : payments) System.out.println(p);
        System.out.println("\n--- CANCELLATIONS ---");
        for (Cancellation c : cancellations) System.out.println(c);
        System.out.println("\n--- REVIEWS ---");
        for (Review r : reviews) System.out.println(r);
        System.out.println("\n--- LISTING RATINGS ---");
        for (Listing l : listings) {
            int total = 0, count = 0;
            for (Review r : reviews) {
                if (r.getBooking().getListing().getId() == l.getId()) {
                    total += r.getRating();
                    count++;
                }
            }
            if (count > 0) {
                System.out.printf("%s: %.2f/5 (%d reviews)%n",
                        l.getName(), (double) total / count, count);
            } else {
                System.out.println(l.getName() + ": No reviews yet");
            }
        }
    }
}
```

src/app/Main.java

```
package app;

import service.HomestayService;
import java.util.Scanner;
import java.time.LocalDate;
import java.time.format.DateTimeParseException;

public class Main {
    static Scanner sc = new Scanner(System.in);
    static HomestayService service = new HomestayService();

    static String read(String msg) {
        System.out.print(msg);
        return sc.nextLine().trim();
    }

    static int number(String msg) {
        return Integer.parseInt(read(msg));
    }

    static LocalDate date(String msg) {
        return LocalDate.parse(read(msg + " (YYYY-MM-DD): "));
    }

    public static void main(String[] args) {
        while (true) {
            System.out.println("\n=== HOMESTAY BOOKING & REVIEWS ===");
            System.out.println("1. Add Listing");
            System.out.println("2. Add Host");
            System.out.println("3. Add Guest");
            System.out.println("4. Create Booking");
            System.out.println("5. Record Payment");
            System.out.println("6. Cancel Booking");
            System.out.println("7. Add Review");
            System.out.println("8. Display Availability");
            System.out.println("9. Display All Records");
            System.out.println("10. Exit");

            try {
                int choice = number("Enter choice: ");
                switch (choice) {
                    case 1:
                        service.addListing(read("Listing name: "),
                                read("Location: "),
                                Double.parseDouble(read("Price per night: ")),
                                number("Host ID: "));
                        break;
                    case 2:
                        service.addHost(read("Host name: "), read("Phone: "));
                        break;
                    case 3:
                        service.addGuest(read("Guest name: "), read("Phone: "));
                        break;
                    case 4:
                        service.createBooking(number("Listing ID: "),
                                number("Guest ID: "),
                                date("Check-in date"),
                                date("Check-out date"));
                        break;
                    case 5:
                        service.recordPayment(number("Booking ID: "),
                                number("1. Cash  2. UPI: "));
                        break;
                    case 6:
                        service.cancelBooking(number("Booking ID: "), read("Reason: "));
                        break;
                    case 7:
                        service.addReview(number("Booking ID: "),
                                number("Rating (1-5): "), read("Comment: "));
                        break;
                    case 8:
                        service.displayAvailability(date("Check-in date"),
                                date("Check-out date"));
                        break;
                    case 9:
                        service.displayRecords();
                        break;
                    case 10:
                        System.out.println("Thank you for using the system!");
```

```
                    sc.close();
                    return;
                default:
                    System.out.println("Choose a number from 1 to 10.");
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid number. Please try again.");
        } catch (DateTimeParseException e) {
            System.out.println("Invalid date. Use YYYY-MM-DD.");
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
}
```